🛠️ TALLER DE CÓDIGO EN EL AULA

Prácticas Guiadas
Objetos en JavaScript

Cuaderno práctico para principiantes: dos proyectos sencillos y guiados paso a paso para aprender a modelar datos del mundo real, vincular métodos con `this` al DOM y gestionar colecciones con arrays de objetos.

🎓 Práctica 1: La Ficha del Alumno

Crea el objeto `alumno`, implementa métodos para calcular su media y estado, y actualiza la tarjeta interactiva en el DOM.

🛒 Práctica 2: Mini-Catálogo de Productos

Trabaja con un array de objetos, renderiza tarjetas de productos, gestiona el botón "Vender 1 unidad" y añade nuevos productos.

Ramón Abramo
Profesor de Desarrollo Web y Programación Frontend · Santander (Cantabria)

Certificado de Profesionalidad IFCD0110
Edición 2026 · Soluciones Oficiales Comentadas

📑 ÍNDICE GENERAL DE LAS PRÁCTICAS

(Haz clic en cualquier sección para saltar a ella)

📦 Portada Oficial del Taller	Pág. 1	📋 Instrucciones Generales para el Alumno	Pág. 2
🎓 Práctica 1: Ficha del Alumno (Enunciado)	Pág. 3	🧭 Práctica 1: Guía y Solución Comentada	Pág. 4
🛒 Práctica 2: Catálogo de Productos (Enunciado)	Pág. 5	🧭 Práctica 2: Guía y Solución Comentada	Pág. 6

📋 Instrucciones Generales para el Alumno

El objetivo de este taller es **aprender la esencia de los objetos en JavaScript sin perderse en complejidades innecesarias**. Cada ejercicio se enfoca en resolver una necesidad web real mediante código directo, limpio y fácil de entender.

📂 Estructura de cada carpeta de práctica:

- `index.html` : Contiene la interfaz gráfica terminada (formulario, botones y tarjetas).
- `style.css` : Contiene el diseño visual preparado con colores y tipografía moderna.
- `app.js` : **Tu archivo de trabajo**. Viene estructurado con comentarios paso a paso y marcas `// TODO:` para que escribas tu código.
- `appSolucion.js` : **Solución oficial resuelta por el profesor** para comprobar tu resultado cuando termines.

💡 Metodología de Trabajo Recomendada

PASO	ACCIÓN EN EL CÓDIGO	CONSEJO DEL PROFESOR RAMÓN
1. Leer el Enunciado	Identifica qué propiedades y métodos necesita el objeto antes de escribir código.	Dibuja en un papel el objeto: qué datos guarda y qué acciones debe realizar.
2. Probar en Consola	Abre la consola de Chrome (F12) y haz `console.log(objeto)` para ver cómo se comporta.	Comprueba que los métodos devuelven lo esperado antes de tocar el DOM.
3. Conectar al DOM	Usa `document.getElementById()` y actualiza `textContent` o `innerHTML`.	Separa la lógica del objeto de la actualización visual de la pantalla.

> 💡 **¿Cómo alternar entre tu código y la solución?**
> En `index.html`, al final del archivo verás: `<script src="app.js"></script>`. Si deseas ver la solución oficial en directo, solo cambia `app.js` por `appSolucion.js` y recarga el navegador.

🎓 Práctica 1: Ficha Interactiva del Alumno

Objetivo Didáctico: Modelar la entidad `alumno` mediante un objeto literal con datos numéricos y de texto, incorporar métodos de cálculo mediante la palabra clave `this` y reflejar dinámicamente los cambios en la pantalla (DOM).

📋 Enunciado del Ejercicio:

En una academia de formación web en Santander, queremos crear una pequeña ficha interactiva para gestionar el progreso de un alumno. Se debe:

1. Crear un objeto literal llamado `alumno` con las siguientes propiedades iniciales:
   - `nombre` : "Marcos García"
   - `modulo` : "Confección y Publicación Web"
   - `nota1` : 6.5
   - `nota2` : 7.5
2. Añadir al objeto un método `calcularMedia()` que sume `this.nota1` y `this.nota2` y divida entre 2, devolviendo el resultado con un decimal (usando `toFixed(1)`).
3. Añadir un método `obtenerEstado()` que devuelva `"Aprobado ✅"` si su media es mayor o igual a 5, o `"Suspenso ❌"` si es menor.
4. Al cargar la página o pulsar el botón **"Mostrar Ficha"**, pintar los datos en los elementos HTML correspondientes (`#txtNombre` , `#txtModulo` , `#txtMedia` , `#txtEstado`).
5. Al pulsar el botón **"Subir 1 punto a Nota 2"**, incrementar `alumno.nota2` en 1 (con tope máximo de 10) y actualizar automáticamente la ficha en pantalla.

🖥️ Maquetación HTML de Referencia

```
<!-- Estructura HTML en index.html -->
<div class="tarjeta-alumno">
  <h2 id="txtNombre">——</h2>
  <p><strong>Módulo:</strong> <span id="txtModulo">——</span></p>
  <p><strong>Notas Parciales:</strong> <span id="txtNotas">——</span></p>
  <p><strong>Media Final:</strong> <span id="txtMedia">——</span></p>
  <p><strong>Estado:</strong> <span id="txtEstado">——</span></p>

  <div class="botones">
    <button id="btnMostrar">Mostrar Ficha</button>
    <button id="btnSubirNota">Subir 1 punto a Nota 2</button>
  </div>
</div>
```

🎯 Criterio de Evaluación:

El cálculo de la media debe hacerse **dentro del propio objeto** a través de su método y no con cálculos sueltos en el evento del botón. Esto garantiza una correcta encapsulación orientada a objetos.

🧭 Guía Paso a Paso y Solución Oficial Comentada

Analiza con atención el código de la solución. Observa cómo el método `renderizar()` centraliza la actualización de la pantalla:

```
// ═══════════════════════════════════════════════════════
// SOLUCIÓN OFICIAL: Práctica 1 - Ficha del Alumno
// Profesor Ramón Abramo · Santander 2026
// ═══════════════════════════════════════════════════════

// 1. Capturamos los elementos del DOM
const txtNombre = document.getElementById("txtNombre");
const txtModulo = document.getElementById("txtModulo");
const txtNotas  = document.getElementById("txtNotas");
const txtMedia  = document.getElementById("txtMedia");
const txtEstado = document.getElementById("txtEstado");
const btnMostrar = document.getElementById("btnMostrar");
const btnSubirNota = document.getElementById("btnSubirNota");

// 2. Definición del objeto literal con propiedades y métodos
const alumno = {
  nombre: "Marcos García",
  modulo: "Confección y Publicación Web",
  nota1: 6.5,
  nota2: 7.5,

  // Método para calcular la media usando 'this'
  calcularMedia() {
    const promedio = (this.nota1 + this.nota2) / 2;
    return promedio.toFixed(1);
  },

  // Método para saber si está aprobado o suspenso
  obtenerEstado() {
    return this.calcularMedia() >= 5 ? "Aprobado ✅" : "Suspenso ❌";
  }
};

// 3. Función auxiliar para pintar los datos del objeto en pantalla
function renderizarFicha() {
  txtNombre.textContent = alumno.nombre;
  txtModulo.textContent = alumno.modulo;
  txtNotas.textContent  = alumno.nota1 + " y " + alumno.nota2;
  txtMedia.textContent  = alumno.calcularMedia();
  txtEstado.textContent = alumno.obtenerEstado();
}

// 4. Eventos de los botones
btnMostrar.addEventListener("click", () => {
  renderizarFicha();
});

btnSubirNota.addEventListener("click", () => {
  if (alumno.nota2 < 10) {
    alumno.nota2 = Math.min(10, alumno.nota2 + 1);
    renderizarFicha(); // Repintamos con el nuevo valor
  } else {
    alert("¡La nota ya ha alcanzado el máximo de 10!");
  }
});

// Renderizado inicial automático al cargar
renderizarFicha();
```

💡 **Claves didácticas a recordar:**
- Fíjate en cómo `obtenerEstado()` llama a su propio método vecino mediante `this.calcularMedia()`.
- Modificar la propiedad `alumno.nota2` altera la memoria del objeto; al llamar de nuevo a `renderizarFicha()` la pantalla muestra el nuevo cálculo automáticamente.

🛒 Práctica 2: Mini-Catálogo de Productos (Array de Objetos)

Objetivo Didáctico: Gestionar una colección de entidades homogéneas mediante un **Array de Objetos Literales**, recorrerlo con `for ... of` para generar tarjetas en el DOM, modificar el stock de un objeto específico y permitir añadir nuevos objetos a la lista.

📋 Enunciado del Ejercicio:

Queremos simular el panel de inventario de una pequeña tienda de papelería en Santander. Se pide:

1. Crear un array llamado `productos` con 3 objetos literales iniciales:
   - `{ id: 1, nombre: "Cuaderno A4", precio: 2.50, stock: 5 }`
   - `{ id: 2, nombre: "Bolígrafo Azul", precio: 1.20, stock: 12 }`
   - `{ id: 3, nombre: "Mochila Escolar", precio: 24.90, stock: 2 }`
2. Crear una función `renderizarCatalogo()` que recorra el array con un bucle `for ... of` y genere dinámicamente en un contenedor `#contenedorProductos` el HTML de cada tarjeta.
3. Cada tarjeta debe mostrar: nombre, precio, stock restante y un botón **"Vender 1 unidad"**.
4. Al pulsar "Vender", se buscará el producto correspondiente por su `id` y se le restará 1 unidad a su stock. Si el stock llega a 0, el botón debe deshabilitarse y mostrar el texto *"Agotado"*.
5. Añadir un formulario con inputs de nombre, precio y stock para que el usuario pueda registrar un nuevo producto en el catálogo mediante `productos.push( ... )`.

🖥️ Maquetación HTML de Referencia

```html
<!-- Formulario de alta rápida -->
<form id="formNuevoProducto">
  <input type="text" id="inputNombre" placeholder="Nombre del producto" required>
  <input type="number" id="inputPrecio" step="0.01" placeholder="Precio (€)" required>
  <input type="number" id="inputStock" placeholder="Stock" required>
  <button type="submit">Añadir al Catálogo</button>
</form>

<!-- Contenedor donde se inyectan las tarjetas -->
<div id="contenedorProductos" class="grid-productos"></div>
```

🎯 Criterio de Evaluación:

Cada vez que se añade un producto o se reduce el stock, se debe volver a llamar a `renderizarCatalogo()` para que la vista del DOM se mantenga perfectamente sincronizada con los datos del array.

🧭 Guía Paso a Paso y Solución Oficial Comentada

Observa cómo asociamos el evento de vender a cada botón mediante un atributo `data-id` para identificar qué objeto debe modificar su stock:

```
// ═══════════════════════════════════════════════════════════
// SOLUCIÓN OFICIAL: Práctica 2 - Catálogo de Productos
// Profesor Ramón Abramo · Santander 2026
// ═══════════════════════════════════════════════════════════

const contenedor = document.getElementById("contenedorProductos");
const formNuevo  = document.getElementById("formNuevoProducto");

// 1. Array inicial de objetos
const productos = [
  { id: 1, nombre: "Cuaderno A4", precio: 2.50, stock: 5 },
  { id: 2, nombre: "Bolígrafo Azul", precio: 1.20, stock: 12 },
  { id: 3, nombre: "Mochila Escolar", precio: 24.90, stock: 2 }
];

// 2. Función para renderizar las tarjetas
function renderizarCatalogo() {
  contenedor.innerHTML = ""; // Limpiamos el contenedor

  for (const prod of productos) {
    const agotado = prod.stock === 0;
    const tarjeta = document.createElement("div");
    tarjeta.className = "tarjeta-producto";

    tarjeta.innerHTML = `
      <h3>\${prod.nombre}</h3>
      <p>Precio: <strong>\${prod.precio.toFixed(2)}€</strong></p>
      <p>Stock: \${prod.stock} uds.</p>
      <button class="btn-vender" data-id="\${prod.id}" \${agotado ? "disabled" : ""}>
        \${agotado ? "Agotado" : "Vender 1 ud."}
      </button>
    `;

    contenedor.appendChild(tarjeta);
  }

  // Asociamos los clics a todos los botones de venta
  document.querySelectorAll(".btn-vender").forEach(btn => {
    btn.addEventListener("click", (e) => {
      const idProducto = Number(e.target.dataset.id);
      venderProducto(idProducto);
    });
  });
}

// 3. Función para vender una unidad
function venderProducto(id) {
  const prod = productos.find(item => item.id === id);
  if (prod && prod.stock > 0) {
    prod.stock--;
    renderizarCatalogo(); // Actualizamos la vista
  }
}

// 4. Manejo del formulario de alta rápida
formNuevo.addEventListener("submit", (e) => {
  e.preventDefault();
  const nuevo = {
    id: Date.now(),
    nombre: document.getElementById("inputNombre").value.trim(),
    precio: parseFloat(document.getElementById("inputPrecio").value),
    stock: parseInt(document.getElementById("inputStock").value)
  };

  productos.push(nuevo);
  formNuevo.reset();
  renderizarCatalogo();
});

renderizarCatalogo();
```

> 💡 **Claves didácticas a recordar:**
> - Cada producto tiene su propia identidad (`id`), lo que permite localizarlo con `.find()` .
> - Añadir un nuevo objeto al array con `productos.push()` es directo y sencillo.